UNITY ASSET STORE PACKAGE

CryoController

User guide

Version 1.0.0 · Unity 6 (6000.0 or newer) · Input System 1.8 or newer

A complete, drop-in first person character: tight CharacterController movement with coyote time and jump buffering, acceleration and air control, crouching, power slides, sprinting with stamina, slope sliding, mantling, ladders, swimming, moving platforms, rigidbody pushing, picking up, inspecting and throwing objects, leaning round corners, head bob, landing dip, strafe tilt, zoom, camera shake and kicks, mouse, gamepad and touch controls with response curves, an interaction system with doors, drawers, locks, keys and readable notes, a flashlight, surface-aware footstep and landing audio, fall damage, saving and loading, a pause menu with input rebinding, a controls overlay, saved player preferences, and movement profiles to swap the whole feel at once. Everything is plain C# on top of the built-in CharacterController and the Input System package, and works in every render pipeline.

Contents

1. Requirements
2. Getting started in 60 seconds
3. What is in the package
4. Input
5. Components
6. Building levels
7. Pause menu and preferences
8. Scripting
9. Saving and loading
10. Render pipelines
11. Tests
12. Troubleshooting
13. Support

Requirements

- Unity 6 (6000.0 or newer).
- The Input System package (com.unity.inputsystem) 1.8 or newer, with **Active Input Handling** set to *Input System Package* or *Both* (Edit > Project Settings > Player > Other Settings).
- No other packages. Everything, the demo scene included, works in the Built-in Render Pipeline, URP and HDRP.

Getting started in 60 seconds

1. **Import the package.** If the Input System package is not installed yet, install it from Window > Package Manager (Unity Registry > Input System), and let Unity restart when it offers to enable the new input backend.

2. **Try the demo.** Open `Demo/Playground` and press Play. Move with WASD, look with the mouse, Space jumps, Shift sprints, C crouches, Q and E lean, F interacts and picks things up, R inspects what you carry, the left mouse button throws, the right mouse button zooms, Escape pauses. 1 and 2 switch between the movement profiles. A gamepad works out of the box too (the D-pad switches profiles). Climb the tower and jump off for fall damage, and use the detonator by the boxes for a camera shake.
3. **Add the player to your scene.** Delete the scene's Main Camera (the player has its own), then choose GameObject > CryoController > First Person Player: it lands on the ground under the Scene view. Or drag `Prefabs/Player` in yourself and place it with its feet on the ground.
4. **Make it yours.** Select the player. **Walk Speed**, **Sprint Speed**, **Jump Height** and **Mouse Sensitivity** on the First Person Controller are the settings most games change first. For a different feel in one go, drop a profile from `Profiles` (Realistic, Arcade or Tactical) into its **Profile** field. Every setting has a tooltip, and Components lists them all.

The prefab ships with its own input actions (`Input/First Person Actions`), so it plays in any project without touching the project-wide actions. See Input to use your own bindings.

What is in the package

Folder	Contents
`Prefabs/Player`	The player: capsule, camera, controller and every companion component, ready to drop in.
`Scripts/Player`	`FirstPersonController`, `MovementProfile` and the companion components (audio, rumble, fall damage, interaction, carrying, keys, notes, HUD, controls overlay, held item, flashlight, stamina, preferences, pause menu, touch controls).
`Scripts/Interaction`	`IInteractable`, `IHoldInteractable`, `Interactable`, `ToggleInteractable`, `Door`, `KeyPickup`, `Readable`, `Carryable`, `InteractableHighlight` and `Ladder`.
`Scripts/Audio`	`FootstepSurface` assets and the `FootstepSurfaceMarker` component.
`Scripts/Save`	`SaveSystem`, the `ISaveable` interface and the optional `SaveId` component.
`Scripts/Level`	`MovingPlatform`, a kinematic mover for lifts, walkways and turntables, and `WaterVolume` for water to swim in.
`Scripts/Editor`	The GameObject > CryoController menu, and Save Settings As Movement Profile in the First Person Controller's component menu.
`Profiles`	Movement profiles: Default (the prefab's own settings), Realistic, Arcade and Tactical.
`Input/First Person Actions`	Input actions asset with full keyboard and mouse and gamepad bindings, and a few touch, joystick and XR bindings to build on.
`Audio`	Placeholder footstep, jump and landing clips, plus wood and metal footsteps and the `Surfaces` assets that use them. Replace them with your own.

Folder	Contents
`Demo/Materials`, `Demo/Textures`, `Demo/Scripts`	The demo's grid texture, its materials in Built-in, URP and HDRP versions and its sky; `DemoEnvironment`, which picks the right materials and lays the grid out in metres; and the demo's profile switcher, health bar and detonator. Demo only.
`Demo/Playground`	A test level with a prototype grid: a staircase, blocks, a pushable crate, two boxes to carry and throw, ramps, a lever and gate, a mantle ledge, two ladders, a tower to jump off, a detonator, a moving platform and a turntable.
`Tests`	Automated play mode and Edit mode tests for the Unity Test Framework.
`Third-Party Notices.txt`	The licences of the fonts embedded in the PDF guide.
`Documentation`	This guide, as PDF and Markdown.

All scripts live in the `CryoController` namespace and the `CryoController` assembly, and appear under **Add Component > CryoController**.

Input

The controller reads these actions from an `InputActionAsset`, all in an action map called **Player**:

Action	Type	Keyboard and mouse	Gamepad
Move	Vector2	W A S D or arrow keys	Left stick
Look	Vector2	Mouse	Right stick
Jump	Button	Space	A (south)
Sprint	Button	Left Shift (hold)	Left stick press
Crouch	Button	C (hold, or toggle)	B (east)
Interact	Button	F	Y (north)
Zoom	Button	Right mouse button (hold)	Left trigger (hold)
LeanLeft	Button	Q	Left shoulder
LeanRight	Button	E	Right shoulder
Throw	Button	Left mouse button	Right trigger
Inspect	Button	R	D-pad up
Flashlight	Button	L	D-pad down

Pause (Escape or Start), the menu keys and the cursor keys (Escape releases the mouse, a click locks it again) are read straight from the devices and need no action, as are F1 and the View button for the controls overlay. Zoom, LeanLeft, LeanRight, Throw, Inspect and Flashlight are not in Unity's project template asset, which also binds Interact to E: with that asset, zoom, lean and inspecting are off and Attack throws (see Troubleshooting).

Which asset is used. The controller's **Actions Override** field decides:

- Assigned (the prefab assigns `Input/First Person Actions`): that asset.
- Empty: the project-wide actions asset (`InputSystem.actions`, Edit > Project Settings > Input System Package). Unity's project templates ship one with the same Player map, so the controller works there unchanged.

Either way the controller works on a private runtime copy of the asset: it enables the actions it uses on the copy while it is enabled, and rebinding changes the copy, never the asset in the project nor the project-wide asset other scripts share. `FirstPersonController.Actions` is the copy.

Rebinding. The pause menu's Controls page lets the player change the keyboard and mouse or gamepad binding of the actions listed in its **Rebindable Actions** (Move, Jump, Sprint, Crouch, Interact, Zoom, LeanLeft, LeanRight, Throw and Inspect by default). Changes are binding overrides on the asset in use, saved by Player Preferences and restored on start; Reset bindings clears them. `InputRebinder` does the listing and rebinding, so a custom menu can offer the same.

To rebind, edit the bindings in whichever asset the controller uses. Move and Look are required: without them the controller disables itself and says so in the Console. Jump, Sprint and Crouch are optional; a missing one is reported once and that feature is off. Interact is required by the Player Interactor only. The asset also carries Attack, Previous and Next actions for your game; the controller does not use them (the demo switches movement profiles with Previous and Next).

Components

All numbers below are the defaults on the prefab. Every field has a tooltip in the Inspector.

First Person Controller

Requires a `CharacterController` on the same object and a `Camera` on a child; the camera is the head and turns with the look input. Runs in `Update`, so movement is smooth at any frame rate and in step with the input.

Profile

Field	Default	Purpose
Profile	none	A Movement Profile whose settings replace the movement, acceleration, jump and gravity, slope, mantling, ladder, crouch speed, power slide, head bob, camera feel, zoom and lean settings below while the game runs.

Movement (metres per second)

Field	Default	Purpose
Walk Speed	4	Speed while walking.
Sprint Speed	7	Speed while Sprint is held. Not possible while crouched.
Crouch Speed	2	Speed while crouched.

Acceleration (metres per second squared)

Field	Default	Purpose
Ground Acceleration	40	How quickly the target speed is reached on the ground. Higher is snappier.
Ground Deceleration	60	How quickly the player stops with no input.
Air Control	6	How quickly input can change the velocity in the air. Ground momentum is kept and there is no air drag.

Jump and gravity

Field	Default	Purpose
Jump Height	1.2	Peak height of a jump, in metres.
Gravity	20	Downward acceleration in m/s². Stronger than real gravity feels less floaty.
Coyote Time	0.12	Seconds after walking off a ledge during which a jump is still accepted.
Jump Buffer Time	0.12	Seconds before landing during which a jump press is remembered and performed on landing.
Air Jumps	0	Extra jumps in mid-air before landing: 1 is a double jump. A press in the air after coyote time uses one.
Air Jump Height	1	How high an air jump rises from where it starts, in metres. It cancels any fall but never slows a faster rise.

Slopes

Field	Default	Purpose
Max Slide Speed	8	Top speed when sliding down a surface steeper than the CharacterController's Slope Limit.
Slide Control	0.25	How much input still steers while sliding: 0 none, 1 full walking control.

Mantling

Field	Default	Purpose
Mantle Max Height	1.6	Ledges up to this high above the feet can be climbed by pushing forward at them while jumping or falling past them. 0 disables.
Auto Mantle Height	0.9	Ledges up to this high are climbed just by walking into them, once the player is up against them. Ledges below the Step Offset are stepped up.
Mantle Duration	0.45	Seconds a climb takes at Mantle Max Height; lower ledges are proportionally quicker.

Ladders

Field	Default	Purpose
Climb Speed	3	Climbing speed, in metres per second.
Ladder Dismount Speed	3	Speed the player is pushed away when jumping off a ladder.

Swimming

Field	Default	Purpose
Swim Speed	3	Swimming speed in a Water Volume, in metres per second. Sprint swims faster, by Sprint Speed over Walk Speed. 0 never swims.
Swim Acceleration	6	How quickly the swimming speed changes, in m/s².
Swim Depth	1.1	How deep the water must be at the feet before the player swims; shallower water is waded through.
Float Height	0.15	How far the eyes stay above the surface while floating.
Climb Out Height	0.7	The highest edge above the water the player climbs out onto by swimming into it.

Look

Field	Default	Purpose
Mouse Sensitivity	0.05	Degrees per pixel of mouse or touch movement.
Mouse Smoothing	0	Frames the look delta is averaged over. 0 is raw input; 2 to 4 softens jitter at low frame rates.
Stick Sensitivity	150	Degrees per second at full stick deflection.
Max Pitch	85	How far up or down the camera can look.
Invert Y	off	Invert vertical look.
Stick Curve Exponent	2	Stick response curve: 1 is linear, 2 keeps small deflections slow for fine aiming.
Stick Vertical Scale	0.7	Vertical stick speed relative to horizontal.
Stick Acceleration Boost	2	Turn speed multiplier reached when the stick is held at full deflection. 1 disables.
Stick Acceleration Delay	0.25	Seconds of full deflection before the boost starts.
Stick Acceleration Ramp	0.75	Seconds over which the boost ramps up.

Crouch

Field	Default	Purpose
Crouch Height	1	Capsule height while crouched. Standing height comes from the CharacterController.

Field	Default	Purpose
Crouch Transition Speed	6	How fast the capsule shrinks or grows, in metres per second.
Crouch Toggle	off	Off: crouch while held. On: each press toggles. Standing up waits for headroom either way.

Power slide

Field	Default	Purpose
Power Slide	on	Crouching while sprinting on the ground starts a slide: the player drops low and keeps their momentum.
Power Slide Boost	2	Speed added when a slide starts, in m/s.
Power Slide Friction	6	Deceleration on flat ground, in m/s². Slopes add or take away gravity's share, so a slide down a ramp speeds up.
Power Slide Min Speed	2.5	The slide ends below this speed, leaving the player crouched.
Power Slide Steering	45	How fast a slide turns at full sideways input, in degrees per second.

A slide also ends when the crouch is released (or toggled off), on leaving the ground, on a steep slope, or against a wall. Jumping out of a slide keeps its speed. With the defaults a slide from full sprint covers about 6 m in a second.

Head bob

Field	Default	Purpose
Head Bob	on	Bob the camera while walking. Fades in and out with movement.
Head Bob Amplitude	0.03	Vertical amplitude in metres, one bounce per step.
Head Bob Sway	0.02	Sideways amplitude, one sway per two steps (a figure of eight).
Head Bob Step Length	2	Metres per step; also the interval of the `Footstep` event. Speed follows movement speed.

Camera feel

Field	Default	Purpose
Landing Dip	0.015	Camera dip on landing, in metres per m/s of impact speed above 4 m/s. 0 disables.
Smooth Steps	on	Ease the camera up over stairs instead of snapping when the capsule steps up.
Sprint Fov Boost	6	Field of view added while sprinting. 0 disables.
Strafe Tilt	1.5	Camera roll at walking speed sideways, in degrees, leaning into the strafe. 0 disables.

Field	Default	Purpose
Reduce Camera Motion	off	Comfort setting for players prone to motion sickness: no head bob, landing dip, strafe tilt, lean roll or sprint field-of-view boost, and camera shakes scaled by Reduced Motion Shake. Players can switch it in the pause menu.

Camera shake

Field	Default	Purpose
Shake Angle	3	Camera rotation at full shake, in degrees: pitch and yaw up to this, roll up to half.
Shake Offset	0.03	Camera movement at full shake, in metres.
Shake Frequency	15	How fast a shake jitters, in cycles per second: low is a slow sway, high a rattle.
Shake Recovery	1.5	Shake lost per second, where 1 is a full shake.
Reduced Motion Shake	0	How much of a shake is left with Reduce Camera Motion on: 0 none, 1 all of it.

Shakes come from `AddCameraShake(amount)`, from your own scripts (explosions, impacts, weapons) and from Player Fall Damage. The amount adds to `CameraShake`, capped at 1, and dies away by **Shake Recovery** per second. The camera moves by the square of it, so 0.2 is a tremor and 1 a blast. The shake swings the view around where the player is looking without changing it, so the view settles back exactly where it was.

Lean

Field	Default	Purpose
Lean Distance	0.35	How far the head moves sideways at full lean, in metres, to peek round corners. 0 disables.
Lean Angle	12	Camera roll at full lean, in degrees. The head swings about a pivot low in the body, so it also dips a little.
Lean Time	0.12	Seconds a lean takes to reach its side.
Lean Toggle	off	Off: lean while the key is held. On: each press leans to that side, or back upright if already there.

The head never goes into a wall: a sweep from the upright eye towards the lean side stops it short. Sprinting, power sliding, mantling and climbing straighten the player up. The camera, and with it the interaction sweep and a carried object, move with the lean.

Zoom

Field	Default	Purpose
Zoom Fov Scale	0.6	Field of view while Zoom is held, as a fraction of the normal one. Look sensitivity scales with it and sprinting stops. 1 disables.

Field	Default	Purpose
Zoom Time	0.15	Seconds the zoom blends in or out.

Physics and cursor

Field	Default	Purpose
Push Force	40	Force in newtons applied to rigidbodies the player walks into. 0 disables pushing, and loose props can then be climbed.
Environment Layers	all but Ignore Raycast	Layers the controller's own checks treat as solid: ground, ledges, headroom, lean walls and what holds a carried object back. The CharacterController still collides by the layer matrix.
Manage Cursor	on	Lock and hide the cursor while enabled. Escape releases it, a click locks it again.
Controller Only	off	Gamepad-only play: the mouse and other pointers are ignored (no mouse look, no clicks, in the game and the pause menu) and the pointer stays hidden. The keyboard still works. Players can switch it in the pause menu's options.

Movement Profile

An asset holding a set of the controller's movement and camera-feel settings: speeds, acceleration, jump and gravity, slopes, mantling, ladders, crouch speed, power slides, head bob, camera feel, zoom and lean. Assign one to the controller's **Profile** to use it instead of the controller's own settings. Player preferences (look sensitivity, invert Y, field of view, Reduce Camera Motion, Controller Only and the toggle options), the crouch height, push force, layers and cursor stay on the controller.

The package ships four in `Profiles`:

Profile	Feel
Default	The Player prefab's own settings, as a starting point to copy.
Realistic	Slower and weightier: gentle acceleration, little air control, a lower jump, slower climbs, a stronger head bob, no power slides.
Arcade	Fast and responsive: quick acceleration, strong air control, a higher jump plus a double jump, long power slides, quick mantles and climbs, a bigger sprint field-of-view kick.
Tactical	Deliberate and precise: moderate speeds, quick crouching, a subtle head bob and tilt, a narrow zoom and a wide, quick lean.

Make your own with Assets > Create > CryoController > Movement Profile, or save a controller's current settings with **Save Settings As Movement Profile...** in the First Person Controller's component menu (the ⋮ button on its header). In Play Mode that saves the settings in use, so a feel tuned while playing is kept after Unity undoes the Play Mode changes. Edits to the profile in use show straight away in Play Mode.

Swap profiles at runtime through `FirstPersonController.Profile`: another profile while the player is wounded, carries something heavy or wades through water, and null to go back to the settings authored on the controller. A swap takes effect at once; motion already under way, such as the rise of a jump, carries on from where it is.

Player Audio

Plays footsteps (on the controller's `Footstep` event), jumps and landings. **Source** is optional: leave it empty for a 2D source created at runtime, or assign one routed to a mixer group. Footstep volume scales with crouching (**Crouch Volume Scale**) and sprinting (**Sprint Volume Scale**); landing volume rises from **Land Volume Soft** to **Land Volume Hard** as the impact speed approaches **Hard Landing Speed** (a 1.2 m jump lands at about 6.9 m/s). **Pitch Variation** randomises every sound a little. **Slide Clips** play when a power slide starts. Empty clip lists play nothing.

Surfaces lists `FootstepSurface` assets with their own sounds; the first one matching the ground under the player plays instead of the default clips, and **Ladder Surface** plays for rung footsteps while climbing. See Footstep surfaces.

Player Rumble

Gamepad rumble: a pulse on landing that grows with the impact (**Landing Strength** at **Hard Landing Speed**, lasting **Landing Duration**), short pulses when a mantle starts (**Mantle Strength**) and when a carried object is thrown (**Throw Strength**, with Player Carry), each **Pulse Duration** long, and a light buzz while power sliding (**Slide Strength**). Only the gamepad the player is using rumbles, never while input is off, the game is paused or the window is out of focus (unless the Input System's **Background Behavior** is Ignore Focus, so the game still gets input), and the motors stop when the component is disabled. `Pulse(low, high, seconds)` starts a pulse of your own, for weapons or damage, say; pulses that overlap add up, each motor running at the stronger of the two until the later one ends.

Player Fall Damage

Turns hard landings into damage for your own health system; the package keeps no health itself. Landings up to **Safe Speed** (11 m/s, about a 3 m drop at the default gravity) do nothing. Faster ones do damage that scales up to **Max Damage** (100) at **Lethal Speed** (20 m/s, about 10 m), and add a camera shake of up to **Camera Shake** to match. Wire **On Fall Damage** in the Inspector to your health component, or subscribe to `FallDamaged(damage, impactSpeed)` in code. `DamageFor(impactSpeed)` gives the damage a landing would do. The prefab has it with nothing wired, so hard landings only shake the camera until you connect it.

Footstep Surface and Footstep Surface Marker

A `FootstepSurface` asset (Create > CryoController > Footstep Surface) holds **Footstep Clips**, **Land Clips** (either empty falls back to the Player Audio defaults) and a **Volume Scale**, plus how to recognise the ground: **Physics Materials**, **Tags** or **Terrain Layers**. A `FootstepSurfaceMarker` component on a ground object names its surface directly, for its children too, and wins over those rules.

Moving Platform

A ready-made kinematic mover: the platform travels **Travel** metres from where it starts and back at **Speed**, pausing **Wait At Ends** seconds at each end, and turns at **Turn Speed** degrees per second around its up axis. Needs a `Rigidbody` (set kinematic and interpolated for you) and a collider. The player rides any moving ground, so this component is a convenience, not a requirement; see Moving platforms.

Player Interactor

Each frame casts a ray from the camera along its view, out to **Interact Range**. When the ray hits no `IInteractable`, a thin sphere (**Interact Radius**) is swept the same way, so small objects are easy to aim at, while the one right under the crosshair still wins over a bigger one beside it (a key lying in an open drawer). The interactable hit, if it allows it, becomes `Current`, and a completed Interact input uses it. **Interact Layers** are the layers the cast and sweep hit; anything hit blocks the reach, so a wall hides what is behind it. In the shipped actions Interact is a plain press. An object can ask for a hold instead: one that implements `IHoldInteractable` with a `HoldDuration` above zero (an `Interactable` or a `Door` with **Hold Duration** set) is used only once the button has been held that long, and `HoldProgress` reports how far along the hold is for the HUD's bar. A hold belongs to the object it started on: looking away starts it over. For other objects the action's own interaction decides, so a Hold added to Interact in the actions asset applies to them.

`Notify(text)` shows the player a short message for a few seconds, such as "It's locked." or "Picked up the shed key."; the prompt HUD draws it below the prompt, and `Message` and the `Notified` event let your own UI do so.

Player Carry

Picks up, carries, inspects, drops and throws `Carryable` rigidbodies. Aim at one and complete Interact to pick it up; it floats in front of the camera, steered there by its velocity every physics step, so it still collides with the world and knocks other things about. Interact again puts it down, Throw throws it along the view, and Inspect brings it close to turn it over.

Field	Default	Purpose
Max Mass	20	Heaviest object the player can pick up, in kg.
Hold Distance	0.6	Gap between the camera and a carried object, in metres. Bigger objects are held further out by their size.
Hold Responsiveness	15	How stiffly the object follows the hold point, per second. Higher is snappier.
Max Hold Speed	15	Top speed at which the object moves towards the hold point, in m/s, on top of the player's own.
Break Distance	1	An object that gets stuck further than this from its hold point, in metres, once it has been held there, is dropped. A newly picked up object has a second to reach the hands.
Throw Impulse	12	Impulse of a throw, in newton seconds: a 1 kg object leaves at 12 m/s, a 4 kg one at 3 m/s.

Field	Default	Purpose
Max Throw Speed	15	Fastest a thrown object leaves the hands, in m/s.
Inspect Distance	0.35	Gap between the camera and an object being inspected, in metres, in place of Hold Distance.
Inspect Mouse Speed	0.3	Degrees an inspected object turns per pixel of mouse movement.
Inspect Stick Speed	180	Degrees per second an inspected object turns at full stick deflection.

While carried, the object does not collide with the player, has no gravity, and its colliders are on the built-in **Ignore Raycast** layer, which the controller's probes and the interaction sweep leave out: a box held at chest height is not a ledge to mantle onto. All of that is restored when it is let go, and collision with the player comes back once the object is clear of the capsule. Walking up to a wall presses the object against it rather than dropping it. The object is let go when it gets stuck too far behind, when the player grabs a ladder, or when it is switched off, destroyed or made kinematic. Objects the player stands on cannot be picked up.

Throw uses the Player/Throw action, or Player/Attack when the asset has none. A click that re-locks a freed cursor never throws.

Inspecting. Inspect (R, or the D-pad's up) while carrying an object whose **Inspectable** is on brings it to **Inspect Distance** and holds the player still, crouched if they were: movement, look, jump, crouch, zoom and lean stop, through the controller's `MoveAndLookEnabled`, and the look input turns the object instead, the side facing the player following the mouse or stick. Inspect again goes back to carrying it, turned as the player left it. Interact still puts it down, and Throw waits until the inspection is over. The Interaction Prompt HUD shows the object's **Description** meanwhile. Inspecting needs a Player/Inspect action; without one it is off and the Console says so once.

Interaction Prompt HUD

IMGUI feedback for the interactor and the carrier: a centre dot with a ring round it while something can be used, prompts with their button drawn as a key cap (`F` Open) on a rounded backing, a hold progress bar (**Accent Color**), and `F` Drop, `LMB` Throw, `R` Inspect while carrying. While inspecting, the dot goes, the object's description shows on a card and the prompt becomes `Mouse` Turn, `R` Done, `F` Drop. The buttons show the binding for the device in use, keyboard and mouse or gamepad, and follow rebinding; set **Key Hint** to show fixed text instead. **Font Size** is for a 1080-line screen, and everything scales with the screen height. Hidden while the controller's input is off, for example under the pause menu. Replace it with your own UI by listening to the interactor's `FocusChanged` event and the carrier's `PickedUp`, `Dropped`, `InspectionStarted` and `InspectionEnded` events.

Player Stamina

Sprinting drains **Sprint Drain Per Second** from **Max Stamina**; it regenerates at **Regen Per Second** after **Regen Delay** seconds without sprinting. Running dry blocks sprinting until stamina is back above **Recover Fraction** of the maximum. **Show Bar** draws a small rounded bar at the bottom of the screen, scaled with the screen height, while stamina is not full (**Bar Color**, **Exhausted Color**). Remove the component for unlimited sprinting.

Player Preferences

Saves mouse and stick sensitivity, invert Y, field of view, reduced camera motion, controller only and the input binding overrides with `PlayerPrefs`, loading them on start (**Load On Start**). The pause menu's Options and Controls pages edit them; your own menu can set the properties, rebind through `InputRebinder` and call `Save()`. `ResetToDefaults()` restores the look values authored on the controller and `ResetBindings()` the bindings of the actions asset.

Pause Menu

Escape or Start pauses: time scale 0, audio paused, player input off, cursor freed, and an IMGUI menu with Resume, Options (when a Player Preferences component is present), Controls and Quit. The menu is a rounded panel (**Panel Color**) over the dimmed game, with switches for toggles, slider tracks, key caps for bindings and the select and back buttons of the device in use along the bottom; **Accent Color** marks the selected item, switches that are on and slider fills, and **Font Size** is for a 1080-line screen and scales with the screen height. The menu works with the mouse, the keyboard (arrows or W/S move the selection, Enter or Space activates, left and right adjust sliders and toggles, Escape resumes) and a gamepad (D-pad or left stick, A activates, B goes back). With a **Save Slot** set, the main page also has Save, and Load once that slot has a save: they save and load the scene through `SaveSystem` and tell the player with a message. The Controls page lists the bindings of **Rebindable Actions** for the keyboard and mouse or the gamepad (the Device row switches); activating one waits for the next key or button on those devices; Escape (keyboard) or Start (gamepad) cancels. Picking a key or button another listed action already uses swaps the two, so one press never does two things. **Pause On Focus Loss** pauses builds when the window loses focus. Pausing remembers whether the controller's input was on and the cursor locked, and resuming restores exactly that, so a cutscene that paused stays a cutscene. With the controller's **Controller Only** on, the menu ignores the mouse and the pointer stays hidden. In a web build the browser keeps Escape for itself and frees the mouse with it, so the game pauses when the browser takes the mouse back, and there is no Quit button, as a page can't quit. The drawing is easy to replace: keep `Pause()`, `Resume()`, `Back()`, `IsPaused` and the `Paused` and `Resumed` events and draw your own UI.

Controls Overlay

A panel of the controls in the bottom-left corner: Move, Look, Jump, Sprint, Crouch, Use, Throw, Inspect, Zoom, Lean and Pause, each with its keys as key caps (W A S D, Mouse, Space, ...), or its gamepad buttons while a gamepad is in use, following any rebinding. A key lights up in **Accent Color** while its action is held, so the panel also shows what is being pressed in a trailer or tutorial video. It shows for **Show At Start** seconds when the scene starts (0 starts it hidden), and **Toggle Key** (F1) or the gamepad's View button shows and hides it. It hides under menus, while a note is open and while on-screen controls show. `Show()`, `Hide()`, `Toggle()` and `IsVisible` drive it from your own menu, and `Rows` gives the labels and keys for a panel of your own.

Held Item and Flashlight

`HeldItem` makes something held in view move like it is held: put it on a child of the player's camera, at the pose the model rests in, with the model under it. It lags behind a turn by **Sway Amount** (up to **Max Sway** degrees) and catches up at **Sway Recovery**, bobs **Bob Amount** with the player's steps, and lowers **Lower Distance** out of sight while the hands are busy: climbing, mantling, swimming, carrying or

reading. `ForceLowered` lowers it from a script, for a cutscene or a weapon swap, and `Lowered` reports how far down it is.

`Flashlight` switches a light with the Player/Flashlight action (L, or down on the D-pad). Put it on the held item or any child of the player, with the **Lamp** on it or a child; **Start On** switches it on at the start. It has **On Switched On** and **On Switched Off**, the `Switched` event, `IsOn`, `SetOn` and `Toggle`, and saves whether it is on. The demo's player holds one: try it in the shed. A light's intensity is in each render pipeline's own units, so the demo sets the flashlight's per pipeline the way it sets the sun's.

Touch Controls

On-screen controls for phones and tablets: a stick on the left 40 % of the screen that appears where the thumb lands, look by dragging anywhere else, and buttons for Jump, Crouch, Use, Sprint (a toggle that lets go with the stick), Throw and Inspect (while carrying), and Pause. The stick and buttons drive a virtual gamepad that the player's actions already read through their gamepad bindings, so everything that works with a gamepad works with them; the look drag turns the view directly through `AddLook`. While they show, the controller's `OnScreenControls` is true: the touchscreen itself is left out of the actions, so a finger on a control is never also a look drag or a tap, and the prompt HUD drops its key caps.

The virtual gamepad exists only while the controls show, so it never counts as a connected gamepad otherwise. Under the pause menu only the Pause button stays, and a tap on it resumes.

Show is Automatic (on phones and tablets, and anywhere else once the touchscreen is touched), Always or Never. **Look Sensitivity** is degrees per pixel dragged on a 1080-line screen; **Stick Radius**, **Button Size** and **Opacity** size and fade the controls, which scale with the screen height. `ButtonScreenRect(button)` and `StickArea` give each button's place and where the stick starts, for a layout or tutorial of your own. To try them in the Editor, turn on the Input System's touch simulation (Window > Analysis > Input Debugger > Options > Simulate Touch Input From Mouse or Pen) and set Show to Always.

Interactable and Toggle Interactable

`Interactable` is a ready-made `IInteractable`: a **Prompt**, an optional **Hold Duration** (seconds Interact must be held; 0 uses it on a press), an optional **Single Use** and an **On Interact** UnityEvent to wire in the Inspector without code. It needs a Collider on the object or a child for the player to aim at. `ToggleInteractable` flips the active state of its **Targets** on each use: a lever and a gate, a switch and a light, a door made of two poses.

Door

A door, gate, hatch or drawer the player opens and closes with Interact.

- **Motion: Hinged** swings round the object's up axis by **Open Angle** degrees, so put the component on the hinge (or on a model whose pivot is the hinge) with the door's collider on it or on a child. **Sliding** moves it **Open Distance** metres along its forward axis, for drawers and sliding doors; a negative distance moves it back.
- **Swing Away** (hinged): it swings away from the player who opens it, whichever side they are on. Off, the sign of Open Angle picks the direction.

- **Open Time** is the seconds a full swing or slide takes, eased at both ends. **Start Open** opens it at once when the scene starts. **Hold Duration** asks for Interact to be held, as on an `Interactable`.
- It moves a kinematic Rigidbody (added and set up for you), so loose props in its way are pushed. A player in the way is never pushed: the door waits, `IsBlocked` is true, and it carries on once they move.
- **Lock:** a **Locked** door doesn't open. Using it tries the lock: it raises **On Locked Tried** and shows **Locked Message** ("It's locked.") through the interactor. A player whose **Player Keys** holds the door's **Key Id** unlocks and opens it in one go, and **Consume Key** takes the key from them. The prompt follows: **Open Prompt** or **Close Prompt**, **Locked Prompt** without the key and **Unlock Prompt** with it.
- **Events: On Opening**, **On Closing**, **On Unlocked** and **On Locked Tried**, for sounds and the like; in code `Opening`, `Closing`, `Opened`, `Closed` (when the motion finishes), `Unlocked` and `LockedTried`.
- **Scripting:** `IsOpen`, `OpenAmount` (0 closed to 1 open), `IsMoving`, `IsBlocked`, `IsLocked`, `KeyId`, `SetOpen(open, instant)`, `Open(awayFrom)`, `Unlock()` and `Lock()`. `SetOpen` ignores the lock, so a save system can put every door back as it was, and scripted doors (a gate opened by a lever) can open whatever their lock.

Key Pickup and Player Keys

`PlayerKeys` (on the prefab) is the player's key ring: key ids, with `Has`, `Add`, `Remove`, `Clear`, the `KeyAdded` and `KeyRemoved` events, and **Starting Keys** to begin with. A `KeyPickup` is a key lying in the world: using it ("Take shed key", from **Prompt** and **Display Name**) adds its **Key Id** to the ring, tells the player "Picked up the shed key." and, with **Hide When Taken**, switches itself off. Doors whose Key Id matches then open for that player.

Readable and Player Reader

A `Readable` is a note, letter, sign or book page. Using it ("Read", from **Prompt**) opens its **Text** under its **Title** through the player's `PlayerReader` (on the prefab): the page on a paper card over the dimmed game, with the page number and the button that turns the page or, on the last one, closes the note. A line holding only `---` starts a new page. The player holds still while reading and the game carries on; Interact turns the pages. The note closes by itself if its object is switched off or destroyed.

`PlayerReader` has **Font Size** (for a 1080-line screen, scaled with the screen height), **Paper Color** and **Ink Color**; turn off **Draw Notes** to show them in your own UI through `Opened`, `PageTurned` and `Closed`, `Current` and `Page`, and drive it with `Open(note)`, `NextPage()` and `Close()`. `Readable` has **On Read** and the `Read` event, `HasBeenRead`, and `SetText(title, text)` for notes written at runtime. While a note is open, `PlayerInteractor.Busy` is true: nothing is focused and the prompt HUD hides.

Interactable Highlight

Tints an interactable's renderers while the player aims at it. Put it on the object with the interactable, or on a parent of it, and Player Interactor switches it on and off. **Color** is blended into the object's own colour by **Strength**, fading over **Fade Time**; **Renderers** lists what to tint, all child renderers when empty. It works in every pipeline through a MaterialPropertyBlock per material slot on the base colour, so no material is changed or copied, and it keeps the colour's alpha. Property blocks your own scripts set on those

renderers are kept: only the colour is tinted while lit, and it is put back afterwards. The demo's lever and boxes use it.

Carryable

Makes a Rigidbody something the player can pick up with Player Carry. It is an `IInteractable`, so aiming at it shows its **Prompt** ("Pick up"). **Throwable** off means it can only be put down, and **Inspectable** off that it can't be inspected. **Description** is shown while the player inspects it. **On Picked Up**, **On Inspected**, **On Dropped** and **On Thrown** are UnityEvents for sounds or game logic. Needs a non-kinematic Rigidbody no heavier than the carrier's **Max Mass**, and a collider on the object or a child.

Ladder

Marks a trigger collider as a climbable ladder; see Ladders.

Water Volume

Marks a trigger collider as water to swim in; see Water. **Flow** is a current that carries a swimming player along, in metres per second, and **Underwater Tint** is laid over the screen while the camera is under the surface, in any render pipeline (an alpha of 0 shows nothing).

Building levels

Ground and steps. Anything with a collider works. The CharacterController's **Step Offset** (0.3 m on the prefab) is the tallest step walked up without a mantle, and its **Slope Limit** (45°) the steepest walkable slope. Steeper surfaces slide.

Pushable props. Give a prop a non-kinematic `Rigidbody`. The player shoves it with **Push Force**; heavier props move less. Things the player stands on are never pushed.

Mantling needs nothing in the level: any ledge with a flat top up to **Mantle Max Height** above the feet can be climbed, from knee height (just above the **Step Offset**) up. Walk into ledges up to **Auto Mantle Height**; jump at taller ones while pushing forward. A walk-in climb waits until the player is up against the ledge, so a staircase whose steps are under the **Step Offset** is walked up, not climbed. Pushable props, meaning non-kinematic rigidbodies, are pushed rather than climbed; with **Push Force** at 0 nothing is pushed, so they are climbed like any other ledge. Kinematic ones such as moving platforms can be climbed.

Moving platforms

The player stands on whatever the capsule touches and follows it: each frame the controller moves the player by however far the spot they stood on travelled and turned since the last frame, before their own movement. Nothing is parented, so the capsule never inherits a platform's scale. Any way of moving works: `MovingPlatform`, an Animator, a script writing the transform, or a kinematic Rigidbody moved with `MovePosition`. Give physics-driven platforms **Interpolate** so the pose the player rides is smooth between physics steps.

Leaving moving ground keeps what it was carrying: a jump on a moving platform lands back on it, stepping off a lift keeps its rise or fall (so a jump on a lift is the same jump relative to the lift), and air control steers relative to the platform's motion. Head bob and footsteps count only the player's own movement, so riding a lift is quiet.

Footstep surfaces

1. Create a `FootstepSurface` asset per kind of ground (Create > CryoController > Footstep Surface) and give it footstep and landing clips.
2. Tell it how to recognise its ground: add physics materials, tags or terrain layers to the asset, or put a `FootstepSurfaceMarker` on the ground object and pick the surface there. Terrains are sampled at the player's feet, and the layer with the most weight decides.
3. Add the asset to the **Surfaces** list on Player Audio. The first surface that matches wins; ground that matches none plays the default clips.

The demo's Wood and Metal surfaces are placeholders. Markers mark the Stairs and Ramp as wood and the moving platform and turntable as metal, and Metal is the demo's ladder surface.

Ladders

1. Create an empty object at the ladder and add a `BoxCollider` covering the space the player stands in while climbing: about 0.8 m wide, the ladder's full height, and reaching from the rungs to a little beyond the player's capsule (0.9 m deep works for the prefab).
2. Add the **Ladder** component. It makes the collider a trigger.
3. Rotate the object so its up axis (green gizmo ray) points along the ladder and its forward axis (blue) points from the rungs into the wall, towards the platform the player steps onto at the top.
4. Add the visible rungs and rails as children, offset out of the zone so the player does not collide with them.

Walking into the zone while facing the ladder, or falling into it, starts a climb. Move forward and back climb up and down; reaching the top mantles onto the platform; climbing down onto the ground lets go; Jump pushes the player away from the rungs at **Ladder Dismount Speed**.

Water

1. Add an object with a `BoxCollider` (or any collider) filling the water, from the bottom to the surface: the top of its bounds is the surface.
2. Add the **Water Volume** component. It makes the collider a trigger.
3. Give the water its look with a mesh of your own at the surface, such as a quad facing up and another facing down so the surface shows from below too.

Water shallower than the controller's **Swim Depth** at the player's feet is waded through. Deeper, the player swims: they move where they look, so swimming forward while looking down dives; Jump rises and Crouch dives; and a still player drifts up and floats with the eyes **Float Height** above the surface. Falling into deep water ends the fall without a landing, so it takes no fall damage. To get out, wade up a slope or steps, or swim into an edge up to **Climb Out Height** above the water to climb onto it. Sprint swims faster

without using stamina. `IsSwimming`, `IsUnderwater`, `Water` and the `StartedSwimming` and `StoppedSwimming` events report it.

Interactables

Add `Interactable` (or `ToggleInteractable`) to an object with a collider, set the prompt and wire **On Interact**. Add `InteractableHighlight` too to make it light up when aimed at. For code-driven objects implement `IInteractable` (see Scripting). Objects the player must not reach through, such as glass, only need to be on a layer included in the interactor's **Interact Layers**.

Carryable objects

Add `Carryable` to any non-kinematic Rigidbody with a collider. Keep it within the player's **Max Mass** (20 kg by default); heavier ones show no prompt. A carried object's colliders sit on the Ignore Raycast layer until it is let go, so layer-based logic of your own sees that layer during a carry. Give it a **Description** to show while the player inspects it: a note, a clue, what the object is. The demo's Small Box (1 kg) and Box (4 kg) show how mass changes a throw, and both have one.

Pause menu and preferences

The prefab includes both. While paused the controller's `InputEnabled` is false, so no movement or look leaks through, the cursor is free for the menu, and the interaction HUD and stamina bar hide. Resume locks the cursor again, and the press that resumed (Space, A, B) is not passed on as a jump or crouch. Both components draw with IMGUI so they work with no UI setup at all; to skin them, keep the components for their logic and events and draw your own canvas on `Paused` / `Resumed` and `FocusChanged`.

Preferences and rebound controls are saved when a page is left or the game resumes, under the PlayerPrefs keys `Player.MouseSensitivity`, `Player.StickSensitivity`, `Player.InvertY`, `Player.FieldOfView`, `Player.ReduceCameraMotion`, `Player.ControllerOnly` and `Player.Bindings` (the `PlayerPreferences.*Key` constants).

Scripting

Everything is in `namespace CryoController`. `FirstPersonController` is one `partial` class split by topic: `FirstPersonController.cs` holds the settings, state, events and the frame loop, and `FirstPersonController.Input.cs`, `.Look.cs`, `.Movement.cs`, `.MovingGround.cs`, `.Crouch.cs`, `.Mantle.cs`, `.Ladders.cs`, `.Lean.cs`, `.Camera.cs` and `.Profile.cs` hold the rest.

FirstPersonController

Member	Meaning
`IsGrounded`, `IsCrouching`, `IsSprinting`, `IsSliding`, `IsPowerSliding`, `IsMantling`, `IsClimbing`, `IsSwimming`, `IsUnderwater`, `IsZooming`	Current state (`IsSliding` is the slope slide, `IsPowerSliding` the crouch slide).
`LeanAmount`	How far the player leans: -1 fully left, 0 upright, 1 fully right.
`AirJumpsLeft`	Air jumps left before the player lands again.
`UsingGamepad`, `LastInputDevice`	Whether the last move or look input came from a gamepad, and the device it came from, for button prompts and rumble.
`PressesSuppressed`, `SuppressPresses()`	True for the frames after the cursor locks, the window regains focus or `SuppressPresses()` is called (a menu closing on a press); a component reading its own button presses should skip them.
`Velocity`	World velocity in metres per second, everything included: movement, climbs and moving ground.
`Ground`, `GroundVelocity`	The collider under the player (null in the air) and the velocity it carries the player at, non-zero on moving platforms.
`EnvironmentLayers`	Layers the controller's own checks treat as solid.
`InputEnabled`	Set false to freeze the player for menus, dialogue or cutscenes; gravity still applies and the cursor is left to you.
`MoveAndLookEnabled`	Set false to hold the player still while the game carries on around them: movement and look input stop, but the cursor, prompts and other components keep working. Player Carry clears it while inspecting.
`SprintAllowed`	A veto for sprinting. Player Stamina sets it only when stamina runs out and when it recovers, so a veto of your own set in between stays.
`MouseSensitivity`, `StickSensitivity`, `InvertY`, `FieldOfView`, `ReduceCameraMotion`	Live look and camera settings.
`ControllerOnly`	Gamepad-only play: pointers are left out of `Actions` (no mouse look or clicks for any component using them), the pause menu ignores them, and the pointer stays hidden.
`Profile`	The Movement Profile in use, or null for the settings authored on the controller. Setting it swaps the settings at once. `MovementProfile.CreateFrom(controller)` makes a new profile from a controller's current settings.

Member	Meaning
`Actions`	The `InputActionAsset` in use.
`Jumped`, `Landed(float impactSpeed)`, `Footstep`, `Mantled(float height)`, `PowerSlideStarted`, `PowerSlideEnded`	Events.
`AddCameraShake(float amount)`, `CameraShake`	Shake the camera (0.2 a tremor, 1 a blast; shakes add up), and how much it is shaking now.
`AddLook(Vector2 degrees)`	Turn the view by these angles (x right, y up), as a look input that isn't an action, such as a touch drag.
`OnScreenControls`	True while on-screen controls drive the player; set by Touch Controls.
`AddCameraKick(Vector2 degrees)`	Kick the view up (x) and right (y), as recoil or a hit does; it springs back within about half a second and doesn't move where the player aims. Kicks add up.
`Pitch`, `Yaw`, `LookAt(Vector3 point)`	Read or set where the player looks, in degrees (pitch positive looks down), or turn to face a point, at once.
`LockCursor()`, `UnlockCursor()`	Cursor control, also used by the pause menu.
`Teleport(Vector3 position, float yawDegrees)`	Move the player without the CharacterController fighting it.
`EnterLadderZone(Ladder)`, `ExitLadderZone(Ladder)`	Called by ladders; call them yourself for custom triggers.
`EnterWater(WaterVolume)`, `ExitWater(WaterVolume)`, `Water`, `StartedSwimming`, `StoppedSwimming`	Called by water volumes, the water the player is in, and events for swimming.

Shaking the camera for a nearby explosion, stronger the closer it is:

```
using CryoController;
using UnityEngine;

public class Explosion : MonoBehaviour
{
    [SerializeField] private float shakeRadius = 15f;

    private void Start()
    {
        FirstPersonController player = FindAnyObjectByType<FirstPersonController>();
        float distance = Vector3.Distance(player.transform.position, transform.position);
        player.AddCameraShake(Mathf.Clamp01(1f - distance / shakeRadius));
    }
}
```

Taking fall damage into your own health component:

```
[SerializeField] private PlayerFallDamage fallDamage;
[SerializeField] private Health health;   // yours

private void OnEnable() => fallDamage.FallDamaged += OnFallDamaged;
private void OnDisable() => fallDamage.FallDamaged -= OnFallDamaged;

private void OnFallDamaged(float damage, float impactSpeed) => health.Take(damage);
```

Swapping the feel while the player is hurt:

```
[SerializeField] private FirstPersonController player;
[SerializeField] private MovementProfile wounded;   // slower, no sprint FOV kick, a
heavier bob

public void OnHurt() => player.Profile = wounded;
public void OnHealed() => player.Profile = null;   // back to the settings on the
controller
```

Freezing the player for a cutscene:

```
player.InputEnabled = false;   // movement and look stop; the capsule still settles under
gravity
player.UnlockCursor();         // if the cutscene has UI
// ...
player.LockCursor();
player.InputEnabled = true;
```

Interaction

Implement `IInteractable` for full control, or derive from `Interactable` and override `OnInteract` and `OnFocusChanged`:

```
using CryoController;
using UnityEngine;

public class Collectible : Interactable
{
    protected override void OnInteract(PlayerInteractor interactor)
    {
        Debug.Log("Collected " + name);
        gameObject.SetActive(false);
    }

    protected override void OnFocusChanged(bool focused)
    {
        // highlight while aimed at
    }
}
```

`PlayerInteractor` exposes `Current`, `CurrentPrompt`, `HoldProgress`, `InteractAction`, `Carry` and the events `FocusChanged(IInteractable)` and `Interacted(IInteractable)`. `IgnoreCurrentPress()` makes it ignore the Interact press in progress until release, for a component that uses the same press for something else. `PlayerStamina` exposes `Stamina`, `MaxStamina`, `Normalized`, `IsExhausted`, `Refill()` and the events `Exhausted` and `Recovered`.

Carrying

`PlayerCarry` exposes `IsCarrying`, `Carried`, `CanThrow`, `ThrowAction`, `IsInspecting`, `CanInspect`, `InspectAction`, `CanPickUp(Carryable)`, `PickUp(Carryable)`, `Drop()`, `Throw()`, `StartInspecting()`, `StopInspecting()` and the events `PickedUp`, `Dropped`, `Thrown`, `InspectionStarted` and `InspectionEnded`, each with the `Carryable`. `PlayerCarry.CarriedLayer` is the layer carried objects sit on. `Carryable` exposes `Body`, `Carrier`, `IsCarried`, `Throwable`,

`Inspectable` and `Description`, and its `Description`, `CanInteract` and `OnFocusChanged` can be overridden.

```
using CryoController;
using UnityEngine;

// Hands the player a key when a level starts.
public class GiveKey : MonoBehaviour
{
    [SerializeField] private PlayerCarry carry;
    [SerializeField] private Carryable key;

    private void Start()
    {
        carry.PickUp(key);
    }
}
```

Rebinding from your own menu

```
using CryoController;

InputRebinder rebinder = new InputRebinder();
rebinder.Refresh(player.Actions, new[] { "Jump", "Sprint" }, InputRebinder.KeyboardScheme);
foreach (InputRebinder.Row row in rebinder.Rows)
{
    Debug.Log($"{row.Label}: {row.Display}");   // "Jump: Space"
}
rebinder.StartRebind(rebinder.Rows[0], InputRebinder.KeyboardScheme, () =>
preferences.SaveBindings());
```

`Refresh` lists one row per button binding and per part of a composite (Move Up, Move Down, ...) in the given control scheme (`KeyboardScheme` or `GamepadScheme`). `StartRebind` waits for the next key or button on that scheme's devices, cancels on Escape (keyboard) or Start (gamepad) or after `InputRebinder.Timeout` seconds, and calls back either way; `CancelName(scheme)` gives the cancel control's name for a hint. While `IsRebinding`, ignore your menu's own input, and also on the frame `LastFinishedFrame`, so the press that finished the rebind is not also taken as a menu press.

Saving and loading

`SaveSystem` saves the state of a scene and puts it back, with nothing to set up for the package's own components:

Component	What a save keeps
First Person Controller	Where the player stands and looks (they are restored standing).
Player Stamina, Player Keys	The stamina, and the keys on the ring.
Door	Open or shut, locked or not, and which way it swung.
Key Pickup	Whether the key has been taken.
Carryable	Where it is, how it moves, and whether it is switched on; a carried object is put down first.
Readable	Whether it has been read.
Interactable, Toggle Interactable	Whether a single-use one has been used, and whether each target is switched on.

`SaveSystem.Save(slot)` writes the active scene's state to a file per slot in `Application.persistentDataPath` (IndexedDB in a web build), and `Load(slot)` puts it back, returning false for an empty slot or a save of another scene. `HasSave`, `Delete` and `PathFor` manage the slots, and `Saved` and `Loaded` are raised after each. `Capture(scene)` and `Restore(scene, text)` work with the save as text, to keep it in a save system of your own or send it to a server.

Loading puts back the state of objects that are in the scene: load the scene first, then the save. Each component's state is kept under a key made from where its object is in the hierarchy and its type, so renaming or moving an object in the Editor makes older saves miss it; give objects you may rename a `SaveId` (Reset fills in a new id). Objects created at runtime aren't in the scene file, so they aren't found again: save them yourself.

To save your own components, implement `ISaveable`: `CaptureState()` returns the state as a string, usually `JsonUtility.ToJson` of a small struct, and `RestoreState(string)` puts it back. A subclass of `Interactable` can override `CaptureExtraState` and `RestoreExtraState` instead.

Render pipelines

The scripts touch nothing pipeline-specific, and the prefab's camera is a plain `Camera` with an `AudioListener`. It works as is in the Built-in Render Pipeline, URP and HDRP; each pipeline adds its own camera data when needed.

The demo scene works in all three too. Its **Demo Environment** object gives every level renderer the version of its material made for the active pipeline (`Demo/Materials` holds a Built-in, a URP and an HDRP version of each) and sets the sun's intensity in that pipeline's units: 1.2 in Built-in and URP, 130000 lux in HDRP. It recognises the pipeline by the shader tag its asset declares, so a project's own subclass of the URP or HDRP asset works too. It runs in the Editor as well, so the scene looks right as soon as it

opens, and the materials and the sun's intensity are always saved in their Built-in form, so they do not change in the file with the pipeline of whoever saved it (a pipeline may still add its own data to a light or camera you edit). It also gives every cube in the level texture coordinates in metres, lined up across neighbouring objects, so the grid texture shows one square per metre on every face whatever the cube's size; the generated meshes are never saved, and the scene keeps Unity's cube (turn off **Metre Grid** to see the plain cubes). The scene as shipped holds no pipeline-specific components. In Built-in and URP the sky is the `Demo Sky` procedural skybox, with gradient ambient light and a light haze. In HDRP the sky, fog and exposure come from the project's default volume profile (Project Settings > Graphics

> HDRP Global Settings). The URP and HDRP materials carry no pipeline version data, so each pipeline sets its own up on import, and the ones for other pipelines are never touched.

Tests

The package ships with automated tests for the Unity Test Framework, which Unity 6 projects include by default. Open Window > General > Test Runner and run the **PlayMode** and **EditMode** tabs. They only compile when the Test Framework package is installed, so projects without it are unaffected.

The play mode tests build their own small world and drive the player through a virtual gamepad. They cover:

- settling on the ground, jump height, air jumps, coyote time and the jump buffer;
- head bob, and Reduce Camera Motion switching it off;
- Controller Only: a mouse button bound to Jump does nothing while it is on, the gamepad still jumps, and the mouse works again once it is off;
- camera shake settling back to exactly where the player was looking, and none with Reduce Camera Motion;
- fall damage: none for short drops, damage in proportion to the impact speed and a camera shake for long ones, and Max Damage for very long ones;
- crouching under a ceiling, and power slides;
- the look API: setting pitch and yaw, looking at a point, and a camera kick that springs back;
- climbing a ladder onto its platform, and letting go of one destroyed or switched off mid-climb;
- swimming: floating at the surface, diving and rising, no landing when falling into water, wading through shallow water, and climbing out onto an edge;
- mantling a knee-high ledge, needing a jump for a high one, walking up stairs without mantling, and leaving out ledges on excluded layers and pushable props;
- the interaction highlight, and keeping other scripts' property blocks while it is lit;
- riding a moving platform and a turntable;
- picking up, holding, dropping and throwing objects, a tap picking up and the next tap dropping, and what may not be picked up;
- a hold-to-use object needing the hold, and a tap not using it;
- doors and drawers: opening and closing, swinging away from the player, waiting for a player in the way, locks that open only with their key, and taking a key;
- reading a note: turning its pages, closing it, and closing it when the note goes away;

- saving and loading: putting the player, a door, a key, a box and a lever's target back as saved, slots, and refusing another scene's save;
- the held item lagging behind a turn and lowering while carrying, and the flashlight switching and saving;
- the controls overlay showing at the start, F1 toggling it, and its keys following rebinding;
- touch controls: the stick moving the player, dragging to turn, tapping Jump, and leaving the touchscreen out of the actions;
- inspecting a carried object: the look input turns it while the player holds still, it is held closer, Interact puts it down while Throw waits, a crouch is kept, and only an inspectable object in hand can be inspected;
- leaning, and leaning into a wall;
- pausing and resuming, including leaving a cutscene's input off;
- gamepad rumble on a hard landing, and none while input is off;
- rebinding from a gamepad press, swapping with an action that already used the control, cancelling with Start, and saving and loading bindings;
- movement profiles: applied on start, swapped at runtime and back, and every setting copied both ways;
- playing the demo scene in the project's render pipeline: the player lands on the level, every level renderer has the material made for that pipeline, and the package logs no errors or warnings.

The Edit mode tests check the shipped assets: every action the package uses is bound for keyboard and gamepad, no control drives two actions, the Player prefab has all its components and the shipped actions, and neither the demo scene nor the prefab depends on anything outside the package. The shipped movement profiles are checked too: every profile setting is a controller setting, their values make sense, and Default matches the Player prefab. They also cover how `InputRebinder` lists bindings.

The tests leave the project as they found it. They swap in a temporary Input System settings object so input reaches the game whatever window has focus, add a gamepad only the test actions are bound to, and put back any bindings saved in PlayerPrefs.

Troubleshooting

The player does not move or look. Check that Active Input Handling includes the Input System package, and that the controller's actions asset (Actions Override, or the project-wide asset when it is empty) has a **Player** map with Move and Look actions. Missing actions are reported once in the Console.

Mouse look does nothing in the Editor. The cursor must be locked: click the Game view. Escape releases it.

In a web build, Escape pauses the game. The browser keeps Escape for itself: it frees the mouse without the game seeing the key, so the pause menu opens when the mouse is freed. Click the game to capture the mouse again.

Two audio listeners. Your scene still has a Main Camera; delete it or remove its AudioListener.

The player falls through the floor when placed. Place the prefab with its feet at or slightly above the ground; the capsule's bottom is at the object's position.

Ladders are not grabbed. Check the Ladder gizmo: green must point up the ladder and blue from the rungs into the wall. The zone must reach the player's capsule, and the player must face the ladder when walking in.

Zoom, lean, throwing or inspecting do nothing, or E interacts. The actions asset in use is Unity's project template asset (Actions Override empty), which has no Zoom, LeanLeft, LeanRight, Throw or Inspect actions and binds Interact to E. Assign `Input/First Person Actions` to Actions Override, or add those actions to your asset. Without Throw, Attack throws.

A carried object ignores my raycasts. While carried its colliders are on the Ignore Raycast layer; include that layer in your mask, or check `PlayerCarry.Carried`.

Mantling does not trigger. The ledge top must be flat, within **Mantle Max Height** of the feet, and there must be room for the standing capsule on top.

The player slides off a moving platform. The platform needs a collider the capsule can stand on, and if it is moved by physics, a kinematic Rigidbody moved with `MovePosition` and set to Interpolate. A platform moved by setting `transform.position` in `Update` also works.

Footsteps ignore a surface. Check the order of the **Surfaces** list (the first match wins), that the ground collider itself carries the physics material or tag, and for terrains that the surface lists the painted layer.

Zoom does nothing. The actions asset in use needs a Button action named Zoom in the Player map; the shipped `First Person Actions` has one, Unity's project template asset does not. Add one, or set **Zoom Fov Scale** to 1 to turn the feature and its start-up warning off.

An action is missing from the Controls page. Only button bindings and composite parts in the Keyboard&Mouse or Gamepad control scheme are listed. Check the action's name in **Rebindable Actions** and that its bindings belong to those schemes (the Binding Properties panel in the actions editor).

Support

Questions, bug reports and feature requests: use the publisher contact on the Asset Store page.

Version history

1.0.0 — Initial release.